arXiv:2307.14460v1 [cs.CV] 26 Jul 2023

MiDaS v3.1 – A Model Zoo for Robust Monocular Relative Depth Estimation

Reiner Birkl, Diana Wofk, Matthias Müller

Intel Labs

Abstract

We release MiDaS v3.1[1] *for monocular depth estimation, offering a variety of new models based on different encoder backbones. This release is motivated by the success of transformers in computer vision, with a large variety of pretrained vision transformers now available. We explore how using the most promising vision transformers as image encoders impacts depth estimation quality and runtime of the MiDaS architecture. Our investigation also includes recent convolutional approaches that achieve comparable quality to vision transformers in image classification tasks. While the previous release MiDaS v3.0 solely leverages the vanilla vision transformer ViT, MiDaS v3.1 offers additional models based on BEiT, Swin, SwinV2, Next-ViT and LeViT. These models offer different performance-runtime trade-offs. The best model improves the depth estimation quality by* 28% *while efficient models enable downstream tasks requiring high frame rates. We also describe the general process for integrating new backbones.*

1. Introduction

Monocular depth estimation refers to the task of regressing dense depth solely from a single input image or camera view. Solving this problem has numerous applications in downstream tasks like generative AI [1–3], 3D reconstruction [4–6] and autonomous driving [7, 8]. However, it is particularly challenging to deduce depth information at individual pixels given just a single image, as monocular depth estimation is an under-constrained problem. Significant recent progress in depth estimation can be attributed to learning-based methods. In particular, dataset mixing and scale-and-shift-invariant loss construction has enabled robust and generalizable monocular depth estimation with MiDaS [9]. Since the initial development of that work, there have been several releases of MiDaS offering new models with more powerful backbones [10] and lightweight variants for mobile applications.

[1] github.com/isl-org/MiDaS

Many deep learning models for depth estimation adopt encoder-decoder architectures. In addition to convolutional encoders used in the past, a new category of encoder options has emerged with transformers for computer vision. Originally developed for natural language processing [11] and nowadays the foundation of large language models like ChatGPT [12], transformers have led to a wide variety of new vision encoders since the first vision transformer ViT [13]. Many of these new encoders have surpassed the performance of previous convolutional encoders. Inspired by this, we have identified the most promising transformer-based encoders for depth estimation and incorporated them into MiDaS. Since there have also been attempts to make convolutional encoders competitive [14–16], we also include these for a comprehensive investigation.

The latest release MiDaS v3.1, which is the focus of this paper, offers a large collection of new depth estimation models with various state-of-the-art backbones. The goal of this paper is to describe the integration of these backbones into the MiDaS architecture, to provide a thorough comparison and analysis of the different v3.1 models available, and to provide guidance on how MiDaS can be used with future backbones.

2. Related Work

Monocular depth estimation is inherently an ill-posed problem facing challenges like metric scale ambiguity. Learning-based approaches that aim to directly regress metric depth [17–21] have sought to use supervised training on homogeneous datasets with representative environments (*e.g.*, focusing on indoor or outdoor scenes) to encourage the supervised network to learn an appropriate metric scale. However, this results in overfitting to narrow depth ranges and degrades generalizability across environments. Alternatively, relative depth estimation (RDE) approaches [9, 10, 22] aim to regress pixel-wise depth predictions that are accurate relative to each other but carry no metric meaning. The scale factor and potentially a shift factor remain unknown. By factoring out metric scale, these RDE approaches are able to be supervised through disparity labels, which allows training on combinations of

heterogeneous datasets with varying metric depth scales and camera parameters. This enables improved model generalizability across environments.

The MiDaS family of models originates from a key work in the relative depth estimation space that demonstrated the utility of mixing datasets to achieve superior zero-shot cross-dataset performance [9]. Depth prediction is performed in disparity space (*i.e.*, inverse depth up to scale and shift), and training leverages scale-and-shift-invariant losses to handle ambiguities in ground truth labels. Existing depth estimation datasets are mixed together and complemented with frames and disparity labels from 3D movies, thus forming a large meta-dataset. As MiDaS releases have progressed through several versions, more datasets have been incorporated over time. Datasets are discussed as part of the training overview in Sec. 3.3.

The network structure of MiDaS follows a conventional encoder-decoder structure, where the encoder is based on an image-classification network. The original MiDaS v1.0 and v2.0 models use the ResNet-based [23] multi-scale architecture from Xian *et al*. [24]. A mobile-friendly variant using an EfficientNet-Lite [25] backbone is released as part of MiDaS v2.1. Transformer-based backbones are explored in MiDaS v3.0 [10], where variants of ViT [13] are integrated into the MiDaS architecture to develop Dense Prediction Transformers [10]. This report follows up on these efforts by demonstrating how newer backbones, both convolutional and transformer-based, can be integrated into MiDaS, as well as how depth estimation performance benefits from these novel encoder backbones. Our new models are released as MiDaS v3.1.

3. Methodology

In this section, we first provide a detailed overview of convolutional and transformer-based backbones that we explore when developing models for MiDaS v3.1. We then explain how these encoder backbones are integrated into the MiDaS architecture. Lastly, we describe the training setup and discuss a general strategy for adding new backbones for future extensions.

3.1. Overview of Encoder Backbones

A key guideline for the exploration of new backbones is that the depth estimation quality and compute requirements of alternative encoders in the MiDaS [9] architecture should roughly correlate to their behavior in the original task, which is typically image classification. High quality and low compute requirements are generally mutually exclusive. To cover both tradeoffs for downstream tasks, we have implemented and validated different types of encoders which either provide the highest depth estimation quality or need the least resources.

3.1.1 Published Models

For the release of MiDaS v3.1, we have selected the five encoder types which seem most promising for downstream tasks, either due to their high depth estimation quality or low compute requirements for real time applications. This selection criterion also holds for the different sizes which are usually available for encoder types, like small and large. Our overview therefore splits into three parts: models with new backbones which are part of the MiDaS v3.1 release, models with backbones which were explored but not released and for completeness also the models of earlier MiDaS versions, because some of them are included as legacy models in MiDaS v3.1.

We begin with the new backbones released in MiDaS v3.1, which are all transformer backbones. The highest depth estimation quality is achieved with the BEiT [26] transformer, where we offer the BEiT_{512}-L, BEiT_{384}-L and BEiT_{384}-B variants. The numbers denote the quadratic training resolutions 512x512 and 384x384, while the letters L and B stand for large and base. The BEiT transformer architecture also offers two newer versions, but we did not explore BEiT v2 [27] and BEiT-3 [28]. For BEiT v2 [27] no pretrained checkpoint with a resolution of 384x384 or higher was available, but only checkpoints at 224x224. BEiT-3 [28] was released after we completed the study.

The encoder type yielding the second highest depth estimation quality is the Swin transformer, where we offer models with both Swin [29] and SwinV2 [30] backbones. The available variants with high depth estimation quality are Swin-L, SwinV2-L and SwinV2-B, which are all at the resolution 384x384. For downstream tasks with low compute resources, we also offer a model based on SwinV2-T, with the resolution 256x256 and T denoting tiny. A characteristic of the MiDaS v3.1 models based on the Swin and SwinV2 transformer backbones as provided by the PyTorch Image Models repository [31] is that only quadratic inference resolutions can be used. This is different to other newly released models where the inference resolution may differ from the training resolution.

The last two encoder types released in MiDaS v3.1 are Next-ViT [32] as well as LeViT [33] for low compute downstream tasks. For Next-ViT, we offer a model based on the Next-ViT-L ImageNet-1K-6M encoder at resolution 384x384. For LeViT, there is the variant LeViT-384 at resolution 224x224, which can be used at only quadratic inference resolutions like the Swin transformers. Note that according to the naming convention of the LeViT paper [33] the number 384 in the transformer model name LeViT-384 does not stand for the training resolution but the number of channels in the first stage of the LeViT architecture. As we follow the convention that MiDaS models use the training resolution in the model names, the MiDaS model based on the transformer backbone LeViT-384 is called LeViT_{224}.

3.1.2 Unpublished Models

Next, we give an overview of the backbones explored when developing MiDaS v3.1 that were ultimately rejected due to the resulting depth estimation models being less competitive. This overview includes both transformer and convolutional backbones. For the transformer backbones, we first come back to Next-ViT [32], where we have also tested Next-ViT-L ImageNet-1K. Our exploration also contains a variant of the vanilla vision transformer, which is ViT-L Hybrid. The next type of transformer is DeiT3 [34], where we have explored vanilla DeiT3-L as well as DeiT3-L pretrained on ImageNet-22k and fine-tuned on ImageNet-1K. All these four transformer backbones are at the resolution 384x384. Finally, there is MobileViTv2 [35] for less powerful hardware, where we have implemented the smallest variant MobileViTv2-0.5 at the resolution 256x256 and the largest one, MobileViTv2-2.0 at 384x384. The latter is pretrained on ImageNet-22K and fine-tuned on ImageNet-1K. The numbers 0.5 and 2.0 in the transformer names refer to the width multiplier used in the MobileViTv2 architecture.

We proceed with exploring convolutional backbones, where we consider ConvNeXt [14] and EfficientNet [15]. For ConvNeXt, we have implemented two variants pretrained on ImageNet-22K and fine-tuned on ImageNet-1K, which are ConvNeXt-L and ConvNeXt-XL. For EfficientNet [36], we did not consider any of the base variants EfficientNet-B0 to EfficientNet-B7, but a wider and deeper version of the largest model EfficientNet-B7, which is EfficientNet-L2 [15]. All explored convolutional backbones are at resolution 384x384. However, none of them are in the v3.1 release because they do not result in MiDaS models that yield a sufficiently high depth estimation quality.

3.1.3 Legacy models

For completeness, we also consider the backbones used in previous MiDaS releases. MiDaS v3.0 is based on the vanilla vision transformer [13, 37] backbones ViT-L and ViT-B Hybrid at resolution 384x384. It also contains the convolutional encoders of MiDaS v2.1 as legacy backbones, which are ResNeXt-101 32x8d [38] at 384x384 (=midas_v21_384) and the mobile friendly efficientnet-lite3 [36] at 256x256 (=midas_v21_256_small). These four backbones are included as legacy models in MiDaS v3.1. Earlier backbones are not included, which are the convolutional models ResNeXt-101 32x8d [38] at 384x384 for MiDaS v2.0 and ResNet-50 [39] at 224x224 for MiDaS v1.0. For EfficientNet-Lite3, MiDaS v3.1 also offers an OpenVINO optimized version (=openvino_midas_v21_small_256).

3.2. Integration of Backbones into MiDaS

In the following we provide technical details on how the new backbones released in MiDaS v3.1 are implemented; these are BEiT_{512}-L, BEiT_{384}-L, BEiT_{384}-B, Swin-L, SwinV2-L, SwinV2-B, SwinV2-T, Next-ViT-L ImageNet-1K-6M and LeViT-224 [26, 29, 30, 32, 33]. To minimize the implementation effort, we use the PyTorch Image Models (=timm) repository [31] whenever possible, because this repository offers a common interface to easily exchange backbones. Different backbones are called using a timm function for creating models by providing the name of the desired model. The only exception is Next-ViT, which is not supported by timm but uses it under the hood; we import Next-ViT [32] as an external dependency.

Since the backbones were trained for image classification they do not inherently contain depth estimation functionality. New encoder backbones used in MiDaS are just feature extractors and need to be connected to the depth decoder appropriately. However, all the new backbones share the common property that they process the input image via successive encoding stages similar to the decoding stages present in the depth decoder. Hence, the task of integrating a new backbone is to properly connect encoding and decoding stages by placing appropriate hooks. This means that we take a tensor computed in the encoder and make it available as input for the decoder at one of its stages. This may require extra operators changing the shape of such tensors to fit to the decoder.

3.2.1 BEiT

We begin with the technical details of the BEiT encoder backbones [26]. Getting BEiT transformers instead of the already existing vanilla vision transformers into MiDaS is straightforward, because we can use the timm model creation function mentioned above and use the same hooking mechanism already available in MiDaS v3.0 for ViT [13]. We specify the hooks by providing absolute hook positions with respect to the transformer blocks present in the BEiT encoders. Following the hooks chosen for ViT, we select the absolute hook positions 5, 11, 17, 23 for BEiT_{512}-L and BEiT_{384}-L as well as 2, 5, 8, 11 for BEiT_{384}-B. The intuition behind this choice is that the positions are equidistant with one position being at the last transformer block and a gap at the beginning. In addition to that, connecting the encoder backbone also requires a choice of channels for the connected stages, because all transformer blocks of the new encoders contain the same number of channels whereas the depth decoder has different channel numbers per hierarchy level. Here, we also follow the values available for ViT such that we get 256, 512, 1024, 1024 for the number of channels per stage for BEiT_{512}-L and BEiT_{384}-L as well as 96, 192, 384, 768 for BEiT_{384}-B.

Note that the hook positions and number of channels per stage are based on the MiDaS v3.0 choices and might not be optimal.

There is one important point which makes the implementation of the BEiT transformers in MiDaS v3.1 non-trivial. Although the implementation of BEiT in timm allows arbitrary window sizes, only one such size can be chosen per BEiT encoder created with the timm model creation function. To enable different input resolutions without having to recreate the model, we have modified the original BEiT code of timm by overwriting several timm functions inside of MiDaS. The key problem here is that the variable relative_position_indices, which contains relative position indices, is resolution-dependent. The modification generates new indices whenever an unseen resolution is encountered in a single MiDaS run, which may slightly impact performance; for previously encountered resolutions the already computed indices are reused.

3.2.2 Swin

Similarly, the Swin and SwinV2 transformers [29, 30] also share the same basic implementation in MiDaS v3.1. A key difference to BEiT and ViT, however, is that Swin and SwinV2 are hierarchical encoders, which changes the structure of the transformer blocks. BeiT and ViT encoders are based on a series of vision transformer blocks whose output is a tensor of rank 2, with always the same shape, where one dimension reflects the number of patches (plus 1 for the class token) and the other one is the embedding dimension. In contrast, for the hierarchical encoders, there are successive hierarchy levels, where each level contains multiple transformer blocks. Going down one hierarchy level halves the resolution in each of the two image directions such that the number of patches increases by 4, whereas the size of the embedding space doubles. The output shape of the transformer blocks is therefore constant only within a hierarchy level but not across them. The advantage of this structure is that we can omit some of the operators, like convolutional and fully connected layers, which are used for ViT and BEiT to change the resolution and number of channels for the hooked tensors of the encoder backbone to fit into the depth decoder. Instead, only transpose and unflatten operators are required.

A consequence of the hierarchical structure is that there has to be exactly one hook per hierarchy level, *i.e.*, the hooks cannot be chosen freely. The hooks of the Swin and SwinV2 transformers are therefore provided as relative positions with respect to the first transformer block in a hierarchy level. We choose the positions of the hooks as large as possible to reflect the behavior of ViT and BEiT where the last transformer block is always hooked. We thus get the relative hook positions 1, 1, 17, 1 for all three backbones Swin-L, SwinV2-L and SwinV2-B. Note that we did not perform ablations to evaluate how reasonable this choice is. For the number of channels per hierarchy level, we cannot make a choice but we are forced to the numbers provided by the backbones themselves, which are 192, 384, 768, 1536 for Swin-L and SwinV2-L and 128, 256, 512, 1024 for SwinV2-B.

3.2.3 Next-ViT

The next encoder type is Next-ViT-L ImageNet-1K-6M [32], which is also a hierarchical transformer with 4 stages. Each stage consists of *next* transformer blocks and *next* convolution blocks. Similar to the Swin and SwinV2 transformers, we choose the last block per hierarchy level for the hooks. However, as the implementation of the blocks in Next-ViT is sequential, we do not provide relative hook positions but absolute ones, because this simplifies the implementation. The allowed ranges are 0-2, 3-6, 7-36, 37-39 and we choose the hook positions as 2, 6, 36, 39. The number of channels per hook is again given by the encoder backbone and is this time 96, 256, 512, 1024 (see Table 3 in [32]). A difference to Swin and SwinV2 is that the output tensors of the hooked blocks are tensors of rank 3 and not rank 2, where the resolution in the blocks drops from 96x96 to 12x12 for a square input resolution and the number of channels increases from 96 to 1024. Therefore, no extra operators are required to change the shape of these tensors and they can directly be connected to the depth decoder stages. Note that also non-square resolutions are supported. Another important point is that there is a convolutional stem at the beginning of Next-ViT which does already a part of the encoding from the resolution 384x384 down to 96x96. This can be compared to the convolutional patching in front of for example ViT, which also causes a resolution reduction.

3.2.4 LeViT

A key difference to the previous backbones is that LeViT [33], although also being a hierarchical encoder, is based on only three hierarchy levels. Therefore, we reduce the depth decoder to three hierachy levels for this backbone. To still be able to process images of the resolution 224x224, LeViT-224 utilizes an extra convolutional stem before the attention part, which reduces the resolution to the small value of 14x14. To counter this effect, we insert a similar deconvolutional decoder into the depth decoder. The depth decoder consists of a hierarchical part and a head. The deconvolutional decoder is inserted between these two parts. The convolutional encoder consists of four times the block (Conv2D, BatchNorm2d) with a Hardswish activation function [40] in between each two blocks. For the deconvolutional decoder, we take two (ConvTranspose2D,

BatchNorm2d) blocks with Hardswish in between them and also at the end (kernel size 3 and stride 2 as for the convolutional encoder). Only two instead of four blocks are used, because this is sufficient to get the resolution of the depth maps in MiDaS equal to the input resolution with minimal changes to the depth decoder.

We also have to look at the number of channels per processing stage. The four blocks of the encoder stem increase the 3 RGB channels to $16 \rightarrow 32 \rightarrow 64 \rightarrow 128$. The depth decoder on the other hand has to decrease the number of channels in multiple likewise processing stages. The hierarchical part of the depth decoder has 256 output channels, which is a fixed number across all backbones of MiDaS v3.1, a choice taken over from MiDaS v3.0. For other backbones, this number is successively decreased to $128 \rightarrow 32 \rightarrow 1$, where 1 is the single channel required to represent inverse relative depth. However, for LeViT, the extra deconvolutional decoder already yields a decrease to $128 \rightarrow 64$ at the beginning of the depth decoder head. Therefore, the remaining channel reduction has to be adjusted and we use $32 \rightarrow 8 \rightarrow 1$ to have a gradual decrease.

For the hooks, the situation is similar to the Swin and SwinV2 transformers, where the tensors hooked in the encoder backbone are of rank 2 such that only transposition and unflattening operators are required to get a shape fitting to the depth decoder. The hook positions are absolute and chosen as 3, 11, 21.

3.2.5 Others

The other backbones explored but not released are Next-ViT-L ImageNet-1K, ViT-L Hybrid, vanilla DeiT3-L, DeiT3-L pretrained on ImageNet-22k and fine-tuned on ImageNet-1K, MobileViTv2-0.5, MobileViTv2-2.0, ConvNeXt-L, ConvNeXt-XL and EfficientNet-L2 [13–15, 32, 34, 35]. The first four backbones do not require any new functionality. Next-ViT-L reuses the modifications introduce earlier for Next-ViT-L ImageNet-1K-6M. ViT-L Hybrid is just another variant of ViT-B Hybrid, which is part of MiDaS v3.0. The two DeiT3 backbones are based on the functionality used for ViT. Hence, only MobileViTv2, ConvNeXt and EfficientNet-L require a modification of the MiDaS code. However, this modification is trivial in all these cases, as there are always four hierarchy levels which can directly be hooked into the depth decoder without extra conversion operators. For MobileViTv2, there is not even a free choice in how the hooks can be chosen. For ConvNeXt and EfficientNet-L, we have proceeded similar to the hooking mechanisms explained earlier. The relative hook positions selected for ConvNeXt are 2, 2, 26, 2, with the allowed ranges 0-2, 0-2, 0-26, 0-2; for EfficientNet-L, this choice is 10, 10, 15, 5, with the ranges 0-10, 0-10, 0-15, 0-5.

3.3. Training Setup

We follow the same experimental protocol used in training MiDaS v3.0 [10] that uses multi-objective optimization [41] with Adam [42], setting the learning rate to 1e-5 for updating the encoder backbones and 1e-4 for the decoder. Encoders are initialized with ImageNet [43] weights, whereas decoder weights are initialized randomly. Our training dataset mix is comprised of up to 12 datasets. Similar to [9], we first pretrain models on a subset of the dataset mix for 60 epochs (first training stage), and then train for 60 epochs on the full dataset (second training stage).

Dataset Mix 3+10. This mix is identical to the one used in training MiDaS v3.0. The 10 datasets used include ReDWeb [24], DIML [44], Movies [9], MegaDepth [45], WSVD [46], TartanAir [47], HRWSI [48], ApolloScape [49], BlendedMVS [50], and IRS [51]. A subset consisting of 3 datasets (ReDWeb, HRWSI, BlendedMVS) is used for pretraining models prior to training on the full 10 datasets.

Dataset Mix 5+12. This mix extends the one described above by including NYUDepth v2 [52] and KITTI [53]. These two datasets were kept out of the training mix in earlier versions of MiDaS to enable zero-shot testing. Our decision to include these two datasets in training is motivated by applications where MiDaS is integrated into metric depth estimation pipelines; we observe that additional training data bolsters model generalizability to indoor and outdoor domains in those applications. In experiments that use this extended dataset mix, a subset now consisting of 5 datasets (ReDWeb, HRWSI, BlendedMVS, NYU Depth v2, KITTI) is used for pretraining models prior to training on the full 12 datasets.

3.4. Discussion on using New Backbones

Finally, we describe a general strategy for adding new backbones to the MiDaS architecture for possible future extensions; please refer to Sec. 3.2 for examples. The main steps are as follows. If possible, the PyTorch Image Models repository [31] or a comparable framework should be used to create a new encoder backbone to reduce the implementation effort. This backbone has to be connected to the depth decoder which requires a choice of hook positions in the encoder backbone. Depending on the shape of the tensors used for the hooking, a series of operators may be required to change the shape such that it fits to the corresponding inputs in the depth decoder. If a backbone contains multiple fundamentally different parts like a convolutional stem at the beginning and an attention part afterwards, the easiest approach is to do the hooking only on the attention part, if possible. To get reasonable resolutions during the depth decoding, it may be required to

modify either its hierarchical part or head. This can mean changing the number of hierarchy stages within the network or inverting operators in encoder backbones and inserting them into decoder heads (as we did when integrating the LeViT backbone). Finally, the number of channels at certain network layers may need to be adapted; for this, a helpful guideline may be the structure of similar backbones that have been previously integrated.

4. Experiments

In this section, we describe the evaluation protocol and present a comparison of the various models in MiDaS v3.1 alongside a few legacy models from previous releases. We then cover ablation studies that were performed as we experimented with modifying the backbones being incorporated into MiDaS.

4.1. Evaluation

Models are evaluated on six datasets: DIW [54], ETH3D [55], Sintel [56], KITTI [53], NYU Depth v2 [52] and TUM [57]. The type of error computed for each dataset is given by the choice made in the original MiDaS paper [9]. For DIW, the computed metric is the Weighted Human Disagreement Rate (WHDR). For ETH3D and Sintel, the mean absolute value of the relative error (REL) $\frac{1}{M}\sum_{i=1}^{M}|d_i - d_i^*|/d_i^*$ is used, where M is the number of pixels, d_i is the relative depth and the asterisk, *e.g.*, d_i^*, denotes the ground truth. For the remaining three datasets, the percentage of bad depth pixels δ_1 with $\max(d_i/d_i^*, d_i^*/d_i) > 1.25$ is counted.

For a quick model comparison, we introduce the relative improvement with respect to the largest model ViT-L 384 from MiDaS v3.0. The relative improvement is defined as the relative zero-shot error averaged over the six datasets. Denoting all the errors as ϵ_s, with $s \in \{1, ..., 6\}$ being the dataset index, the improvement is then defined as

$$I = 100\left(1 - \frac{1}{6}\sum_{d}\frac{\epsilon_d}{\epsilon_{d,\mathrm{ViT-L384}}}\right)\% \qquad (1)$$

where $\epsilon_{d,\mathrm{ViT-L384}}$ are the respective errors for the model ViT-L 384. Note that a difference in resolution limits the comparability of the zero-shot errors and thus the improvement. This is because these quantities are averages over the pixels of an image and do not take into account the potential advantage of more details present at higher resolutions. A visualization of the relative improvement versus the frame rate is shown in in Fig. 1.

We also use the root mean square error of the disparity (RMSE) $[\frac{1}{M}\sum_{i=1}^{M}|D_i - D_i^*|^2]^{\frac{1}{2}}$, where D_i is the disparity, for additional comparisons of models during training (cf. Tab. 3).

4.2. Results and Analysis

An overview of the validation results is provided in Tabs. 1, 2 and 3. While Tabs. 1 and 2 show completely trained models, *i.e.*, training is done in two stages, the models in Tab. 3 are not trained beyond the first stage (cf. Sec. 3.3) since the depth estimation quality observed there is too low to justify further training. These models are presented despite incomplete training to show both accepted and discarded backbones. In this section, we discuss the models in Tab. 1, those above the horizontal separator in Tab. 2 and the models between the first and last horizontal separators of Tab. 3. The remaining models are either included for comparisons or they are experimental. A thorough explanation of them can be found in Sec. 4.3.

4.2.1 Published Models

Tab. 1 contains the models released as a part of MiDaS v3.1. BEiT_{512}-L is the best model for both square and unconstrained resolutions. Note that unconstrained resolutions mean an aspect ratio defined by the dataset. The quality of the BEiT_{512}-L model can be seen from the relative improvement I in Tab. 1, which is 36% for square resolutions and 19% for resolutions of height 512 as well as 28% if the height is 384. Note that different inference resolutions have to be considered separately here due to the limitations of the relative improvement I mentioned in Sec. 4.1.

MiDaS v3.1 includes more models than earlier versions to provide a better coverage of possible downstream tasks, including lightweight models. This is reflected by new models like LeViT-224 in Tab. 1, which is the fastest new model with a framerate of 73 frames per second (fps). It is surpassed in speed only by the legacy model EfficientNet-Lite3 that runs at 90 fps.

4.2.2 Unpublished Models

The models in Tab. 2 are not released due to a lower depth estimation quality compared to the released ones. The first of these models is Swin-L, trained on the dataset configuration 3+10. Here, we have released only the variant trained on the configuration 5+12, as shown in Tab. 1. As we see from the rightmost column of Tabs. 1 and 2, the increased number of datasets improves the quality measure I from 2% to 21%, which is a significant jump. The main contribution for this increase comes from KITTI and NYUDepth v2 no longer being zero-shot datasets when trained with the configuration 5+12. This can be seen from the decrease of the δ_1 scores of KITTI and NYUDepth v2 from 12.15 and 6.571 to 6.601 and 3.343 respectively, while the remaining errors decrease only slightly (see Tabs. 1 and 2). The next unreleased model in Tab. 2 is Swin-T, which is

not part of MiDaS v3.1, because SwinV2 generally yields better results than Swin. Finally, we have also studied the MobileViTv2 family of transformers, which contains MobileViTv2-0.5 as our smallest model with 13 million parameters. However, both variants MobileViTv2-0.5 and MobileViTv2-2.0 have values of I around -300%, which reflects a too low quality to be relevant.

As the models below the horizontal separator of Tab. 2 are explained in Sec. 4.3, we proceed with the models between the first and last horizontal separator of Tab. 3. The models shown there split into models with transformer and convolutional encoder backbones, which are separated by the dashed separator. We start with the transformer models, where we first have DeiT3-L-22K-1K and DeiT3-L. These two models have a high depth estimation quality, *e.g.*, 0.070 for the relative error (REL) of the BlendedMVS dataset, which is equal to the value of BEiT$_{384}$-L also visible in Tab. 2 for a comparison. However, as the DeiT3 transformers do not surpass the quality of BEiT$_{384}$-L, we did not train them beyond the first stage. The same criterion holds for ViT-L Hybrid, which was explored, because ViT-B Hybrid is part of MiDaS v3.0 (cf. Tab. 1). For Next-ViT-L-1K and Next-ViT-L-1K-6M, we have decided to include the better of the two variants in MiDaS v3.1, which is Next-ViT-L-1K-6M according to Tab. 3.

Finally, we have also explored the three convolutional models ConvNeXt-XL, ConvNeXt-L and EfficientNet-L2. As we explored them with the intention to get a model of highest quality and it did not beat BEiT$_{384}$-L, we have discarded these models. In particular, EfficientNet-L2 shows a low depth estimation quality with errors of 0.165, 0.227 and 0.219 according to Tab. 3.

4.3. Ablation Studies

In the following, we discuss experimental modifications of some of the investigated backbones, which helps to get a better understanding of the associated configurations. The modifications can be found at the bottom of Tabs. 2 and 3. In addition to that, we also walk through the models at the top of Tab. 2, which are included for a comparison with the other models in that table.

We begin with the four reference models at the top of Tab. 3. Variants of these models are also available in Tab. 1. For BEiT$_{384}$-L and Next-ViT-L-1K-6M, these are models with different training datasets, *i.e.* 3+10 in Tab. 3 and 5+12 in Tab. 1. For Swin-L, no such difference is given between the two tables. However, in Tab. 3, we have included two separate training runs to provide an approximation of the variance in the training process. ViT-L is basically the same model in both tables, but the training runs are independent, because a retraining was required to get the data required for Tab. 3.

We continue with the two experimental modifications at the bottom of Tab. 3, which have undergone only one training stage. The first modification, denoted as ViT-L Reversed, is the vanilla vision transformer backbone ViT-L already released in MiDaS v3.0, but with the order of the hooks reversed. Instead of providing the depth decoder hooks with the absolute positions 5, 11, 17, 23, we set them to 23, 17, 11, 5. This is possible, because the ViT encoder family is based on a series of similar transformer blocks, which do not differ like the transformer blocks in for instance the hierarchical structure of the Swin transformers. Astonishingly, as shown in Tab. 3, the reversal of the hooks has practically no impact on the depth estimation quality. So, there is no major difference if the four hierarchy levels of the decoder are connected in forward or reverse order to the transformer blocks of the encoder.

The second experiment is Swin-L Equidistant where the hooks are chosen as equidistantly as possible, similar to ViT-L. As we consider a Swin transformer here, the hook positions are relative and constrained to 0-1, 0-1, 0-17, 0-1 (cf. Sec. 3.2). To homogenize the distance between the hooks, we replace the positions 1, 1, 17, 1 of Swin-L by 1, 1, 9, 1. Note that the distances could be made even more similar by setting the first hook to zero. However, here we follow ViT-L, where a gap is chosen before the first hook. As we see from Tab. 3, the modification leads to a small decrease of the depth estimation quality when compared to the unmodified model Swin-L such that we have not released the corresponding model. To also get at least a very rough estimate of the significance of this change, we have actually included two independent training runs for Swin-L, denoted by training 1 and 2 in Tab. 3. As we see, the training variance seems to be rather small for Swin-L.

Tab. 2 shows four additional modifications, where we have also trained the second stage. We first consider the model BEiT$_{384}$-L Wide, where the hooks are widened by removing the hook gap at the beginning of the encoder. Instead of the absolute hook positions 5, 11, 17, 23 of BEiT$_{384}$-L in Tab. 1 (see Sec. 3.2), the modification uses 0, 7, 15, 23. As we see from Tab. 2, there is nearly no impact on the depth estimation quality. For unconstrained resolutions, the relative improvement I is 17.4% for the widened variant and thus a bit better than the value 16.8% for the original variant in Tab. 1. For square resolutions, the situation is the opposite, where we have the values 32.7% and 33.0%. With the effect being so small, we have decided to keep the hook gap.

The remaining three modifications in Tab. 2, denoted as BEiT$_{384}$-L 5+12+12K, BEiT$_{384}$-L 5K+12K and BEiT$_{384}$-L 5A+12A, address the large value $\delta_1 = 9.847$ of KITTI for the unconstrained resolution of BEiT$_{384}$-L when compared to $\delta_1 = 2.212$ of NYU Depth v2 in Tab. 1. The reason for the large δ_1 value is that the training images of KITTI have a high aspect ratio caused by the resolution 1280x384, where

Model		Resources		Unconstrained Resolution							Square Resolution						
	Data	Par.	FPS	DIW	ETH3D	Sintel	KITTI	NYU	TUM	I	DIW	ETH3D	Sintel	KITTI	NYU	TUM	I
Encoder/Backbone	Mix	↓	↑	WHDR↓	REL↓	REL↓	δ_1 ↓	δ_1 ↓	δ_1 ↓	%↑	WHDR↓	REL↓	REL↓	δ_1 ↓	δ_1 ↓	δ_1 ↓	%↑
BEiT$_{512}$-L [26]	5+12	345	5.7	0.114	**0.066**	0.237	11.57*	**1.862***	**6.132**	19	0.112	**0.061**	**0.209**	**5.005***	**1.902***	**6.465**	**36**
BEiT$_{384}$-L [26]	5+12	344	13	0.124	0.067	0.255	9.847*	2.212*	7.176	16.8	0.111	0.064	0.222	5.110*	2.229*	7.453	33.0
BEiT$_{512}$-L@384 [26]	5+12	345	5.7	0.125	0.068	**0.218**	**6.283***	2.161*	**6.132**	**28**	0.117	0.070	0.223	6.545*	2.582*	6.804	29
SwinV2-L [30]	5+12	213	41	–	–	–	–	–	–	–	0.111	0.073	0.244	5.840*	2.929*	8.876	25
SwinV2-B [30]	5+12	102	39	–	–	–	–	–	–	–	0.110	0.079	0.240	5.976*	3.284*	8.933	23
Swin-L [29]	5+12	213	49	–	–	–	–	–	–	–	0.113	0.085	0.243	6.601*	3.343*	8.750	21
BEiT$_{384}$-B [26]	5+12	112	31	0.116	0.097	0.290	26.60*	3.919*	9.884	-31	0.114	0.085	0.250	8.180*	3.588*	9.276	16
Next-ViT-L-1K-6M [32]	5+12	72	30	**0.103**	0.095	0.230	6.895*	3.479*	9.215	16	**0.106**	0.093	0.254	8.842*	3.442*	9.831	14
ViT-L [13]	3+10	344	61	0.108	0.089	0.270	8.461	8.318	9.966	0	0.112	0.091	0.286	9.173	8.557	10.16	0
ViT-B Hybrid [13]	3+10	123	61	0.110	0.093	0.274	11.56	8.69	10.89	-10	-	-	-	-	-	-	-
SwinV2-T [30]	5+12	42	64	–	–	–	–	–	–	–	0.121	0.111	0.287	10.13*	5.553*	13.43	-6
ResNeXt-101 [38]	3+10	105	47	0.130	0.116	0.329	16.08	8.71	12.51	-32	-	-	-	-	-	-	-
LeViT-224 [33]	5+12	51	73	–	–	–	–	–	–	–	0.131	0.121	0.315	15.27*	8.642*	18.21	-34
EfficientNet-Lite3 [36]	3+10	**21**	**90**	0.134	0.134	0.337	29.27	13.43	14.53	-75	-	-	-	-	-	-	-

Table 1. **Evaluation of released models (post second training stage).** The table shows the validation of the second training stage (see Sec. 3.3) for the models released in MiDaS v3.1. The dataset definitions 3+10 and 5+12 used for the training can be found in Sec. 3.3. The resources required per model are given by the number of parameters in million (Par.) and the frames per second (FPS, if possible for the unconstrained resolution). The validation is done on the datasets DIW [54], ETH3D [55], Sintel [56], KITTI [53], NYU Depth v2 [52] and TUM [57] with the validation errors as described in Sec. 4.1. The resolution is either unconstrained, *i.e.* the aspect ratio is given by the images in the dataset, or the images are converted to a square resolution. Overall model quality is given by the relative improvement I with respect to ViT-L (cf. Eq. (1)). Note that Next-ViT-L-1K-6M and ResNeXt-101 are short forms of Next-ViT-L ImageNet-1K-6M and ResNeXt-101 32x8d. The suffix @384 means that the model is validated at the inference resolution 384x384 (differing from the training resolution). Legacy models from MiDaS v3.0 and 2.1 are in italics, where ResNeXt-101=midas_v21_384 and Efficientnet-lite3=midas_v21_256_small. Validation errors that could not be evaluated, because of the model not supporting the respective resolution are marked by –. Quantities not evaluated due to other reasons are given by -. The asterisk * refers to non-zero-shot errors, because of the training on KITTI and NYU Depth v2. The rows are ordered such that models with better relative improvement values for the square resolution are at the top. The best numbers per column are bold and second best underlined.

Model		Resources		Unconstrained Resolution							Square Resolution						
	Data	Par.	FPS	DIW	ETH3D	Sintel	KITTI	NYU	TUM	I	DIW	ETH3D	Sintel	KITTI	NYU	TUM	I
Encoder/Backbone	Mix	↓	↑	WHDR↓	REL↓	REL↓	δ_1 ↓	δ_1 ↓	δ_1 ↓	%↑	WHDR↓	REL↓	REL↓	δ_1 ↓	δ_1 ↓	δ_1 ↓	%↑
Swin-L [30]	3+10	213	41	–	–	–	–	–	–	–	0.115	0.086	0.246	12.15	6.571	9.745	2
Swin-T [30]	3+10	42	71	–	–	–	–	–	–	–	0.131	0.120	0.334	15.66	12.69	14.56	-38
MobileViTv2-0.5 [35]	5+12	**13**	**72**	0.430	0.268	0.418	51.77*	45.32*	39.33	-301	0.509	0.263	0.422	37.67*	48.65*	40.63	-286
MobileViTv2-2.0 [35]	5+12	34	61	0.509	0.263	0.422	37.67*	48.65*	40.63	-294	0.501	0.269	0.433	59.94*	48.32*	41.79	-320
BEiT$_{384}$-L 5K+12K	·K	344	13	0.120	0.066	0.213	2.967*	2.235*	**6.570**	35	0.110	0.066	**0.212**	5.929*	2.296*	**6.772**	**33**
BEiT$_{384}$-L Wide	5+12	344	13	0.111	0.068	0.247	10.73*	2.146*	7.217	17.4	0.112	0.066	0.221	**5.078***	2.216*	7.401	**32.7**
BEiT$_{384}$-L 5+12+12K	+12K	344	13	0.123	0.065	0.216	2.967*	2.066*	7.417	33	**0.107**	**0.064**	0.217	5.631*	2.259*	7.659	32
BEiT$_{384}$-L A5+12A	·A	344	13	**0.110**	**0.061**	**0.207**	**2.802***	**1.891***	7.533	**37**	0.113	0.070	0.213	6.504*	**2.179***	7.946	29

Table 2. **Evaluation of unpublished models (post second training stage).** The table shows the validation of the second training stage (see Sec. 3.3) of models not released in MiDaS v3.1 due to a low depth estimation quality. The models below the horizontal separator are based on experimental modifications explained in Sec. 4.3. The general table layout is similar to Tab. 1. The extra dataset mixes, like ·K, are explained in Sec. 4.3.

the width is much bigger than the height. This is different for *e.g.*, NYU Depth v2, where the resolution is 512x384 and thus the aspect ratio is significantly lower. However, in BEiT$_{384}$-L, the resolution 1280x384 is reduced to 384x384 by random cropping such that there is a strong resolution discrepancy between training and inference, because for the unconstrained resolution inference is done with the original resolution 1280x384. In the modifications, we remove this discrepancy by training KITTI on the original resolution 1280x384. Whenever KITTI is trained in this way, we add the letter K as a suffix after the dataset counter. This leads us to the first modification BEiT$_{384}$-L 5+12+12K, where we take the original model BEiT$_{384}$-L trained in two stages on the data 5+12 and add a third stage, which is also trained on the 12 datasets of the second stage but now with the original KITTI resolution. As we see from Tab. 2, this lowers the δ_1 value from 9.847 to 2.967. Note that for simplicity we only provide the dataset change +12K and not the whole description 5+12+12K in the data column of Tab. 1.

For BEiT$_{384}$-L 5K+12K, we use only two training stages

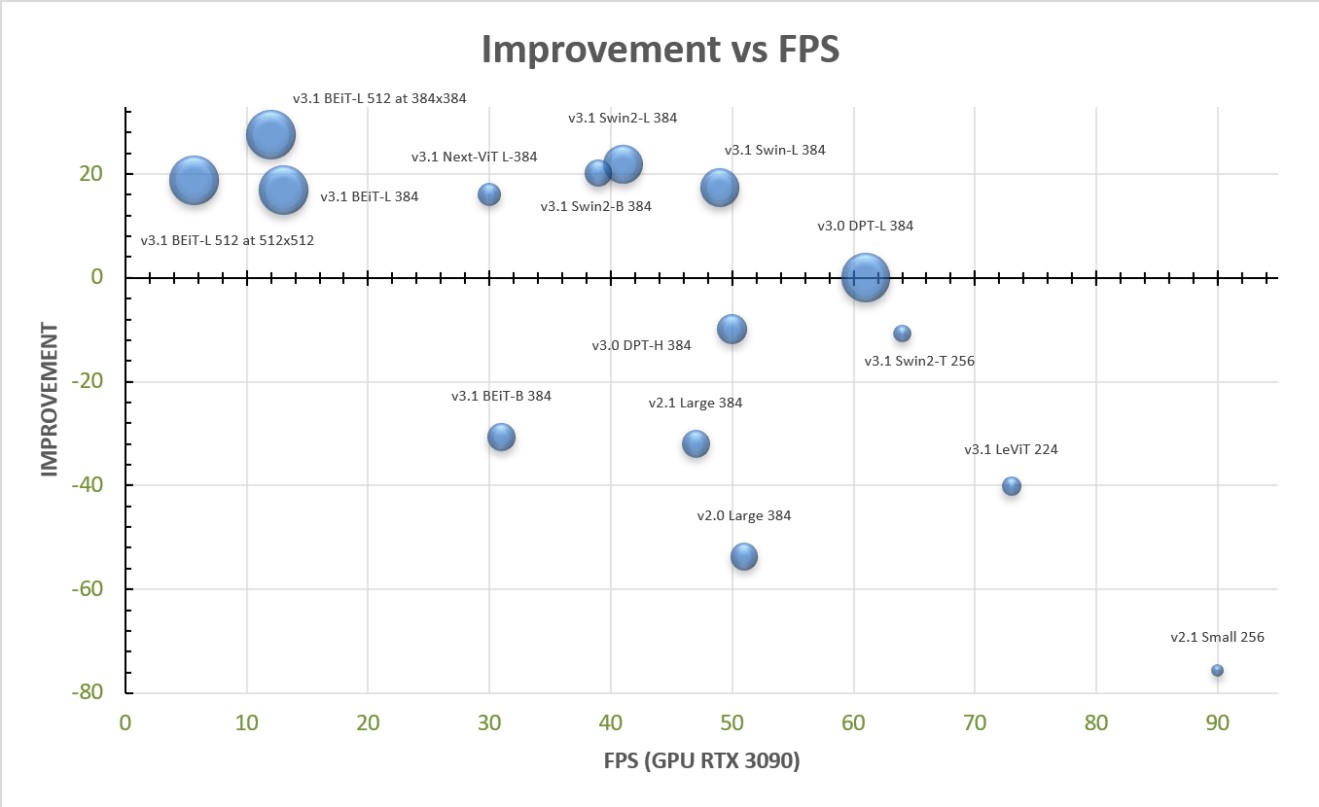

Figure 1. **Improvement vs FPS.** The plot shows the improvement of all the models of MiDaS v3.1 with respect to the largest model $\text{DPT}_{\text{L 384}}$ (=ViT-L 384) of MiDaS v3.0 vs the frames per second. The framerate is measured on an RTX 3090 GPU. The area covered by the bubbles is proportional to the number of parameters of the corresponding models. In the model descriptions, we provide the MiDaS version, because some models of MiDaS v3.1 are legacy models which were already introduced in earlier MiDaS releases. The first 3-digit number in the model name reflects the training resolution which is always a square resolution. For two BEiT models, we also provide the inference resolution at the end of the model description, because there the inference resolution differs from the training one. The improvement is defined as the relative zero-shot error averaged over six datasets as explained in Sec. 4.1.

and train them with the original KITTI resolution. Hence, we denote the dataset as 5K+12K instead of 5+12, or ·K in short. This does not change the δ_1 value of KITTI for the unconstrained resolution, but improves the overall model quality a bit. The relative improvement I increases from 33% to 35% for the unconstrained resolution and 32% to 33% for the square one. We also test extending the approach to use the original aspect ratio of the training images during training for the other datasets. If the training resolution is not constant over the training images, we use the average resolution, adjusted to a multiple of 32%. This gives 480x448 for ReDWeb [24], 480x448 for MegaDepth [45], 384x384 for WSVD [46] and 544x384 for HRWSI [48]. The resulting modified model is BEiT_{384}-L 5A+12A, where the letter A, standing for 'all', denotes that now all training datasets of the respective stage have a resolution close to the original one (·A in the data column of Tab. 2). The consequence of this change is that the δ_1 score of KITTI for the unconstrained resolution drops to the lowest and thus best value 2.802. Also, the relative improvement is best for the modified model, where $I = 37\%$. However, there might be an overfitting to the resolution of the training images, because for square resolutions the relative improvement drops from 33% to 29% and is thus even below the 36% of the BEiT_{512}-L model of Tab. 1. Therefore, we have not released BEiT_{384}-L 5A+12A, but it shows one option for possible future improvements.

5. Applications

The models released as part of the MiDaS v3.1 family demonstrate high relative depth estimation accuracy with successful robustness and generalizability across environments. They are promising candidates for many applications—including architectures that combine relative and metric depth estimation [58,59], architectures for image synthesis [1, 4, 60], and architectures for text-to-RGBD generation [3, 61].

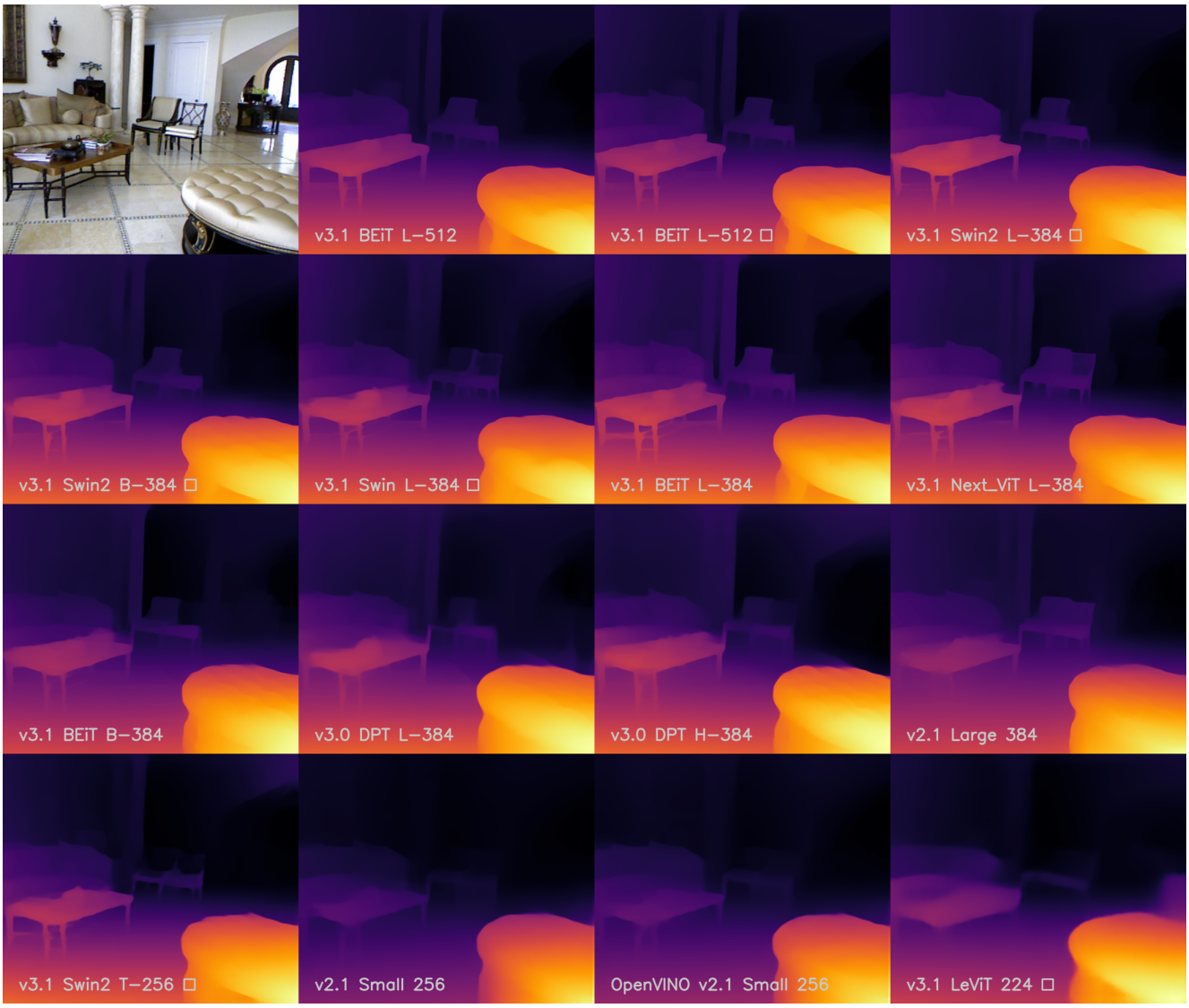

Figure 2. **Backbone comparison.** The table shows the inverse relative depth maps of the different models of MiDaS v3.1, including legacy models, for the example RGB input image at the top, left. The brighter the colors, the larger the inverse relative depths, *i.e.*, the closer the represented objects are to the camera. The names of the models are shown at the bottom left part of each depth map. This includes the MiDaS version, the backbone name and size as well as the training resolution. Models which are evaluated only at a square resolution are marked by the square symbol at the end of the white texts. The second last model at the bottom row is an OpenVINO model.

Metric depth estimation. For practical applications requiring metric depth, MiDaS models on their own are insufficient as their depth outputs are accurate only up to scale and shift. Recent work has shown two approaches to resolving metric scale in depth outputs from MiDaS. Monocular visual-inertial depth estimation [59] integrates generalizable depth models like MiDaS in conjuction with visual-inertial odometry to produce dense depth estimates with metric scale. The proposed pipeline performs global scale and shift alignment of non-metric depth maps against sparse metric depth, followed by learning-based dense alignment. The modular structure of the pipeline allows for different MiDaS models to be integrated, and the approach achieves improved metric depth accuracy when leveraging new MiDaS v3.1 models.

Whereas the above work relies on a combination of visual and inertial data, ZoeDepth [58] seeks to combine relative and metric depth estimation in a purely visual data-driven approach. The flagship model, ZoeD-M12-NK, incorporates a MiDaS v3.1 architecture with the BEiT-L encoder with a newly-proposed metric depth binning module that is appended to the decoder. Training combines

Model	Square Resolution		
	HRWSI RMSE↓	BlendedMVS REL↓	ReDWeb RMSE↓
$BEiT_{384}$-L [26]	**0.068**	**0.070**	**0.076**
Swin-L [29] Training 1	0.0708	0.0724	0.0826
Training 2	0.0713	0.0720	0.0831
ViT-L [13]	0.071	0.072	0.082
Next-ViT-L-1K-6M [32]	0.075	0.073	0.085
DeiT3-L-22K-1K [34]	0.070	**0.070**	0.080
ViT-L Hybrid [13]	0.075	0.075	0.085
Next-ViT-L-1K [32]	0.078	0.075	0.087
DeiT3-L [34]	0.077	0.075	0.087
ConvNeXt-XL [14]	0.075	0.075	0.085
ConvNeXt-L [14]	0.076	0.076	0.087
EfficientNet-L2 [15]	0.165	0.227	0.219
ViT-L Reversed	0.071	0.073	0.081
Swin-L Equidistant	0.072	0.074	0.083

Table 3. **Model evaluation (post first training stage).** The table shows the validation of unpublished models which were mostly trained only in the first training stage and not also the second one due to low depth estimation quality (see Sec. 3.3). The models above the horizontal separator line (between Next-ViT-L-1K-6M and DeiT3-L-22K-1K) are included for a comparison with the other models and have at least a released variant in Tab. 1, although they were also not released directly (see Sec. 4.2 for details). For Swin-L, two different training runs are shown. The models above the dashed separator are models based on transformer backbones, and the models between the dashed and dotted line are convolutional ones. The rows below the dotted separator are models with experimental modifications as explained in Sec. 4.3. All the models in this table are trained on the 3+10 dataset configuration (in contrast to the mixtures of Tabs. 1 and 2). Validation is done on the datasets HRWSI [48], BlendedMVS [50] and ReDWeb [24]. The errors used for validation are the root mean square error of the disparity (RMSE) and the mean absolute value of the relative error (REL), see Sec. 4.1. Note that DeiT3-L-22K-1K is DeiT3-L pretrained on ImageNet-22k and fine-tuned on ImageNet-1K, Next-ViT-L-1K is the shortened form of Next-ViT-L ImageNet-1K and Next-ViT-L-1K-6M stands for Next-ViT-L ImageNet-1K-6M. The model in italics is a retrained legacy model from MiDaS v3.0. The rows are ordered such that better models are at the top. The best numbers per column are bold and second best underlined.

relative depth training for the MiDaS architecture on the 5+12 dataset mix as described in Sec. 3.3, followed by metric depth fine-tuning for the prediction heads in the bins module. Extensive results verify that ZoeDepth models benefit from relative depth training via MiDaS v3.1, enabling finetuning on two metric depth datasets at once (NYU Depth v2 and KITTI) as well as achieving unprecedented zero-shot generalization performance to a diverse set of unseen metric depth datasets.

Depth-conditioned image diffusion. MiDaS has been integrated into Stable Diffusion [1] in order to provide a shape-preserving stable diffusion model for image-to-image generation. Monocular relative depth outputs from MiDaS are used to condition the diffusion model to generate output samples that may vary in artistic style while maintaining semantic shapes seen in the input images. The depth-guided model released as part of Stable Diffusion v2.0 uses DPT-Hybrid from MiDaS v3.0 for monocular depth estimation. It is therefore very promising that MiDaS v3.1 models could be similarly integrated, with their improved depth estimation accuracy allowing for even better structure preservation in image-to-image diffusion.

Joint image and depth diffusion. Ongoing work in the text-to-image diffusion space has motivated the development of a Latent Diffusion Model for 3D (LDM3D) [61] that generates joint image and depth data from a given text prompt. To enable RGBD diffusion, LDM3D leverages a pretrained Stable Diffusion model that is fine-tuned on a dataset of tuples containing a caption, RGB image, and depth map. Training data is sampled from the LAION-400M dataset providing image-caption pairs. Depth maps corresponding to the images are obtained using DPT-Large from MiDaS v3.0. Supervised finetuning enables LDM3D to generate RGB and relative depth map pairs that allows for realistic and immersive 360-degree view generation from text prompts. Utilizing MiDaS v3.1 models to produce depth data for LDM3D finetuning could further improve the quality of LDM3D depth outputs and subsequent scene view generation.

6. Conclusion

We present a collection of robust depth estimation models in the new release MiDaS v3.1. Although we also explore convolutional backbones for the release, only transformer based backbones provide a sufficiently high depth estimation quality with the MiDaS architecture. The release v3.1 consists of depth models with the new transformer backbones BEiT, Swin, SwinV2, Next-ViT and LeViT, where we offer multiple different variants for BEiT and SwinV2. $BEiT_{512}$-L with resolution 512x512 is on average 28% more accurate than MiDaS v3.0 for non-square resolutions. The training of MiDaS has been extended from the original 10 datasets to 12, now including KITTI and NYU Depth V2 using the BTS split [62]. For all of the released backbone types, we provide details on how they are integrated into the MiDaS architecture. We also consolidate this experience into a general guide to how MiDaS may be used with future backbones.

References

[1] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models, 2021. 1, 9, 11

[2] Lvmin Zhang and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. *arXiv preprint arXiv:2302.05543*, 2023. 1

[3] Lukas Höllein, Ang Cao, Andrew Owens, Justin Johnson, and Matthias Nießner. Text2room: Extracting textured 3d meshes from 2d text-to-image models. *arXiv preprint arXiv:2303.11989*, 2023. 1, 9

[4] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021. 1, 9

[5] Shoukang Hu, Kaichen Zhou, Kaiyu Li, Longhui Yu, Lanqing Hong, Tianyang Hu, Zhenguo Li, Gim Hee Lee, and Ziwei Liu. Consistentnerf: Enhancing neural radiance fields with 3d consistency for sparse view synthesis. *arXiv preprint arXiv:2305.11031*, 2023. 1

[6] Shu Chen, Junyao Li, Yang Zhang, and Beiji Zou. Improving neural radiance fields with depth-aware optimization for novel view synthesis. *arXiv preprint arXiv:2304.05218*, 2023. 1

[7] Fei Liu, Zihao Lu, and Xianke Lin. Vision-based environmental perception for autonomous driving. *arXiv preprint arXiv:2212.11453*, 2022. 1

[8] Michaël Fonder, Damien Ernst, and Marc Van Droogenbroeck. M4depth: Monocular depth estimation for autonomous vehicles in unseen environments. *arXiv preprint arXiv:2105.09847*, 2021. 1

[9] René Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. *IEEE transactions on pattern analysis and machine intelligence*, 44(3):1623–1637, 2020. 1, 2, 5, 6

[10] René Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. Vision transformers for dense prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 12179–12188, October 2021. 1, 2, 5

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 1

[12] Yiheng Liu, Tianle Han, Siyuan Ma, Jiayue Zhang, Yuanyuan Yang, Jiaming Tian, Hao He, Antong Li, Mengshen He, Zhengliang Liu, Zihao Wu, Dajiang Zhu, Xiang Li, Ning Qiang, Dingang Shen, Tianming Liu, and Bao Ge. Summary of chatgpt/gpt-4 research and perspective towards the future of large language models, 2023. 1

[13] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. 1, 2, 3, 5, 8, 11

[14] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 1, 3, 5, 11

[15] Qizhe Xie, Minh-Thang Luong, Eduard Hovy, and Quoc V Le. Self-training with noisy student improves imagenet classification. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10687–10698, 2020. 1, 3, 5, 11

[16] Weihao Yu, Mi Luo, Pan Zhou, Chenyang Si, Yichen Zhou, Xinchao Wang, Jiashi Feng, and Shuicheng Yan. Metaformer is actually what you need for vision. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10819–10829, 2022. 1

[17] Shariq Farooq Bhat, Ibraheem Alhashim, and Peter Wonka. Adabins: Depth estimation using adaptive bins. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4009–4018, 2021. 1

[18] Shariq Farooq Bhat, Ibraheem Alhashim, and Peter Wonka. Localbins: Improving depth estimation by learning local distributions. In *European Conference on Computer Vision*, pages 480–496. Springer, 2022. 1

[19] Jinyoung Jun, Jae-Han Lee, Chul Lee, and Chang-Su Kim. Depth map decomposition for monocular depth estimation. *arXiv preprint arXiv:2208.10762*, 2022. 1

[20] Zhenyu Li, Xuyang Wang, Xianming Liu, and Junjun Jiang. Binsformer: Revisiting adaptive bins for monocular depth estimation. *arXiv preprint arXiv:2204.00987*, 2022. 1

[21] Weihao Yuan, Xiaodong Gu, Zuozhuo Dai, Siyu Zhu, and Ping Tan. New crfs: Neural window fully-connected crfs for monocular depth estimation. *arXiv preprint arXiv:2203.01502*, 2022. 1

[22] Jae-Han Lee and Chang-Su Kim. Monocular depth estimation using relative depth maps. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9729–9738, 2019. 1

[23] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016. 2

[24] Ke Xian, Chunhua Shen, Zhiguo Cao, Hao Lu, Yang Xiao, Ruibo Li, and Zhenbo Luo. Monocular relative depth perception with web stereo data supervision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 311–320, 2018. 2, 5, 9, 11

[25] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. 2019. 2

[26] Hangbo Bao, Li Dong, and Furu Wei. Beit: BERT pre-training of image transformers. *CoRR*, abs/2106.08254, 2021. 2, 3, 8, 11

[27] Zhiliang Peng, Li Dong, Hangbo Bao, Qixiang Ye, and Furu Wei. BEiT v2: Masked image modeling with vector-quantized visual tokenizers. 2022. 2

[28] Wenhui Wang, Hangbo Bao, Li Dong, Johan Bjorck, Zhiliang Peng, Qiang Liu, Kriti Aggarwal, Owais Khan Mohammed, Saksham Singhal, Subhojit Som, and Furu Wei. Image as a foreign language: BEiT pretraining for vision and vision-language tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023. 2

[29] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021. 2, 3, 4, 8, 11

[30] Ze Liu, Han Hu, Yutong Lin, Zhuliang Yao, Zhenda Xie, Yixuan Wei, Jia Ning, Yue Cao, Zheng Zhang, Li Dong, Furu Wei, and Baining Guo. Swin transformer v2: Scaling up capacity and resolution. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11999–12009, 2022. 2, 3, 4, 8

[31] Ross Wightman. Pytorch image models. https://github.com/rwightman/pytorch-image-models, 2019. 2, 3, 5

[32] Jiashi Li, Xin Xia, Wei Li, Huixia Li, Xing Wang, Xuefeng Xiao, Rui Wang, Min Zheng, and Xin Pan. Next-vit: Next generation vision transformer for efficient deployment in realistic industrial scenarios. *arXiv preprint arXiv:2207.05501*, 2022. 2, 3, 4, 5, 8, 11

[33] Benjamin Graham, Alaaeldin El-Nouby, Hugo Touvron, Pierre Stock, Armand Joulin, Herve Jegou, and Matthijs Douze. Levit: A vision transformer in convnet's clothing for faster inference. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 12259–12269, October 2021. 2, 3, 4, 8

[34] Hugo Touvron, Matthieu Cord, and Hervé Jégou. Deit iii: Revenge of the vit. In *Computer Vision–ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XXIV*, pages 516–533. Springer, 2022. 3, 5, 11

[35] Sachin Mehta and Mohammad Rastegari. Separable self-attention for mobile vision transformers. *arXiv preprint arXiv:2206.02680*, 2022. 3, 5, 8

[36] Mingxing Tan and Quoc Le. Efficientnet: Rethinking model scaling for convolutional neural networks. In *ICML*, pages 6105–6114. PMLR, 2019. 3, 8

[37] Andreas Steiner, Alexander Kolesnikov, Xiaohua Zhai, Ross Wightman, Jakob Uszkoreit, and Lucas Beyer. How to train your vit? data, augmentation, and regularization in vision transformers. *arXiv preprint arXiv:2106.10270*, 2021. 3

[38] Dhruv Kumar Mahajan, Ross B. Girshick, Vignesh Ramanathan, Kaiming He, Manohar Paluri, Yixuan Li, Ashwin Bharambe, and Laurens van der Maaten. Exploring the limits of weakly supervised pretraining. In *ECCV*, 2018. 3, 8

[39] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, pages 770–778, 2016. 3

[40] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1314–1324, 2019. 4

[41] Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization. In *Advances in Neural Information Processing Systems*, 2018. 5

[42] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations, ICLR*, 2015. 5

[43] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, pages 248–255. Ieee, 2009. 5

[44] Youngjung Kim, Hyungjoo Jung, Dongbo Min, and Kwanghoon Sohn. Deep monocular depth estimation via integration of global and local predictions. *IEEE Transactions on Image Processing*, 27(8):4131–4144, 2018. 5

[45] Zhengqi Li and Noah Snavely. Megadepth: Learning single-view depth prediction from internet photos. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2041–2050, 2018. 5, 9

[46] Chaoyang Wang, Simon Lucey, Federico Perazzi, and Oliver Wang. Web stereo video supervision for depth prediction from dynamic scenes. In *2019 International Conference on 3D Vision (3DV)*, pages 348–357. IEEE, 2019. 5, 9

[47] Wenshan Wang, Delong Zhu, Xiangwei Wang, Yaoyu Hu, Yuheng Qiu, Chen Wang, Yafei Hu, Ashish Kapoor, and Sebastian Scherer. Tartanair: A dataset to push the limits of visual slam. 2020. 5

[48] Ke Xian, Jianming Zhang, Oliver Wang, Long Mai, Zhe Lin, and Zhiguo Cao. Structure-guided ranking loss for single image depth prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 611–620, 2020. 5, 9, 11

[49] Xinyu Huang, Peng Wang, Xinjing Cheng, Dingfu Zhou, Qichuan Geng, and Ruigang Yang. The apolloscape open dataset for autonomous driving and its application. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(10):2702–2719, 2020. 5

[50] Yao Yao, Zixin Luo, Shiwei Li, Jingyang Zhang, Yufan Ren, Lei Zhou, Tian Fang, and Long Quan. Blendedmvs: A large-scale dataset for generalized multi-view stereo networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1790–1799, 2020. 5, 11

[51] Qiang Wang, Shizhen Zheng, Qingsong Yan, Fei Deng, Kaiyong Zhao, and Xiaowen Chu. Irs: A large naturalistic indoor robotics stereo dataset to train deep models for disparity and surface normal estimation. In *2021 IEEE International Conference on Multimedia and Expo (ICME)*, pages 1–6, 2021. 5

[52] Nathan Silberman, Derek Hoiem, Pushmeet Kohli, and Rob Fergus. Indoor segmentation and support inference from rgbd images. In *Computer Vision – ECCV 2012*, pages 746–760, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg. 5, 6, 8

[53] Moritz Menze and Andreas Geiger. Object scene flow for autonomous vehicles. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2015. 5, 6, 8

[54] Weifeng Chen, Zhao Fu, Dawei Yang, and Jia Deng. Single-image depth perception in the wild. *Advances in neural information processing systems*, 29, 2016. 6, 8

[55] Thomas Schops, Johannes L Schonberger, Silvano Galliani, Torsten Sattler, Konrad Schindler, Marc Pollefeys, and Andreas Geiger. A multi-view stereo benchmark with high-resolution images and multi-camera videos. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 3260–3269, 2017. 6, 8

[56] Daniel J Butler, Jonas Wulff, Garrett B Stanley, and Michael J Black. A naturalistic open source movie for optical flow evaluation. In *Computer Vision–ECCV 2012: 12th European Conference on Computer Vision, Florence, Italy, October 7-13, 2012, Proceedings, Part VI 12*, pages 611–625. Springer, 2012. 6, 8

[57] Jürgen Sturm, Nikolas Engelhard, Felix Endres, Wolfram Burgard, and Daniel Cremers. A benchmark for the evaluation of rgb-d slam systems. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pages 573–580. IEEE, 2012. 6, 8

[58] Shariq Farooq Bhat, Reiner Birkl, Diana Wofk, Peter Wonka, and Matthias Müller. Zoedepth: Zero-shot transfer by combining relative and metric depth, 2023. 9, 10

[59] Wofk, Diana and Ranftl, René and Müller, Matthias and Koltun, Vladlen. Monocular Visual-Inertial Depth Estimation. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. 9, 10

[60] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020. 9

[61] Gabriela Ben Melech Stan, Diana Wofk, Scottie Fox, Alex Redden, Will Saxton, Jean Yu, Estelle Aflalo, Shao-Yen Tseng, Fabio Nonato, Matthias Muller, and Vasudev Lal. Ldm3d: Latent diffusion model for 3d. *arXiv preprint arXiv:2305.10853*, 2023. 9, 11

[62] Jin Han Lee, Myung-Kyu Han, Dong Wook Ko, and Il Hong Suh. From big to small: Multi-scale local planar guidance for monocular depth estimation. *arXiv preprint arXiv:1907.10326*, 2019. 11